

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

*I Y. Abhay- 192411219 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Answers:

Question 1 — Student Registration Using Servlet

Scenario

A college wants students to submit registration details through a web form, and a Java Servlet processes the data.

Answer

A **Servlet-based application** follows a client-server architecture. The student enters details in an HTML form. The request is sent to the Servlet, which processes the data and sends a response.

Architecture

Student



HTML Registration Form



Web Browser



Servlet Container



Java Servlet



Database

GET Method

GET is suitable for retrieving non-sensitive information.

```
protected void doGet(HttpServletRequest req,  
                    HttpServletResponse res)
```

```

        throws IOException {

String name = req.getParameter("name");

res.getWriter().println("Student: " + name);
}

```

POST Method

POST is suitable for submitting registration information.

```

protected void doPost(HttpServletRequest req,
                        HttpServletResponse res)
                        throws IOException {

String name = req.getParameter("name");
String email = req.getParameter("email");

res.getWriter().println("Registration Successful");
}

```

Servlet Life Cycle

init() → service() → destroy()

- init() initializes the Servlet.
- service() processes client requests.
- destroy() releases resources before the Servlet is removed.

Which is Better?

For student registration, **POST is more appropriate** because registration details should not be placed in the URL. For real applications, **HTTPS** must also be used to protect the data during transmission.

Conclusion

Therefore, the college registration system can use an HTML form with a **POST request**, and a Servlet can process and store the submitted student details.

Question 2 — Login and Session Management

Scenario

A website requires users to log in and remain authenticated while visiting multiple pages.

Answer

Session management is used to maintain a user's identity across multiple HTTP requests. Since HTTP is stateless, the server needs a mechanism to remember the logged-in user.

Using HttpSession

After successful login:

```
HttpSession session = request.getSession();
```

```
session.setAttribute("username", "student");
```

On another page:

```
HttpSession session = request.getSession();
```

```
String user =
```

```
(String) session.getAttribute("username");
```

```
response.getWriter().println("Welcome " + user);
```

When the user logs out:

```
session.invalidate();
```

Using Cookies

A cookie can store a small piece of information in the browser.

```
Cookie c = new Cookie("username", "student");
```

```
response.addCookie(c);
```

Reading the cookie:

```
Cookie[] cookies = request.getCookies();
```

```
for(Cookie c : cookies) {  
    if(c.getName().equals("username"))  
        response.getWriter().println(c.getValue());  
}
```

Session Working

Login



Server creates Session



Session ID



Browser



Session ID sent with next request



Server identifies user

HttpSession vs Cookies

HttpSession	Cookies
Data mainly maintained on server	Data stored in browser
Suitable for login sessions	Suitable for small preferences
Can be invalidated by server	Controlled by browser
More suitable for sensitive session state	Should not store sensitive information directly

Conclusion

For the login system, **HttpSession is the better choice** for maintaining authentication across pages. Cookies are commonly used to carry the session identifier and can also store non-sensitive preferences.

Question 3 — E-Commerce Product Database Using JDBC

Scenario

An e-commerce website needs to store and retrieve product information using a Java application.

Answer

JDBC (Java Database Connectivity) allows a Java application to communicate with a relational database such as MySQL.

JDBC Architecture

Java Application



JDBC API



JDBC Driver



Database

Steps

1. Import JDBC classes.
2. Establish database connection.
3. Create PreparedStatement.
4. Execute SQL query.
5. Process the result.
6. Close database resources.
7. Handle exceptions.

Database Connection

```
try {
```

```
    Connection con =
```

```
        DriverManager.getConnection(  
            "jdbc:mysql://localhost:3306/shop",  
            "root",  
            "password");
```

```
    System.out.println("Connected");
```

```
} catch(SQLException e) {
```

```
    System.out.println(e.getMessage());
```

```
}
```

Insert Product

```
String sql =
```

```
"INSERT INTO product VALUES(?,?,?)";
```



```
PreparedStatement ps =  
con.prepareStatement(sql);
```

```
ps.setInt(1, 101);  
ps.setString(2, "Laptop");  
ps.setDouble(3, 55000);
```

```
ps.executeUpdate();
```

Retrieve Products

```
Statement st = con.createStatement();
```

```
ResultSet rs =  
st.executeQuery("SELECT * FROM product");
```

```
while(rs.next()) {
```

```
    System.out.println(  
        rs.getInt("id") + " " +  
        rs.getString("name") + " " +  
        rs.getDouble("price"));
```

```
}
```

Exception Handling

Database operations should be placed inside try-catch blocks to handle errors such as connection failure, invalid SQL, or unavailable databases.

PreparedStatement is preferred when user input is involved because it helps prevent **SQL injection**.

Conclusion

Thus, JDBC provides a simple mechanism for the e-commerce application to **connect, insert, retrieve, and manage product data** in a database.
